1 Stochastic Model and Quality Control

This document covers the *measurement* side of the estimator's error budget: the strategies that populate the observation-noise covariance $\boldsymbol{R}$, and the a-posteriori quality-control (QC) layer that validates an epoch's residuals against that $\boldsymbol{R}$. The $\boldsymbol{R}$ produced here is the block-diagonal skeleton of the observation-equations noise model; this document supplies the per-strategy variance interiors of that skeleton. The QC layer is built on the same post-fit residual statistics the filter already computes for outlier rejection, but reframes them as the classical geodetic detection theory of Baarda and Teunissen [3].

Two scopes are deliberately *out* of this document and only cross-referenced. First, the standardised-residual outlier path ($|s| > 6$), the Huber re-weighting, and the EWMA $\boldsymbol{R}$-inflation that react to a detected blunder live in the Kalman-filter gating section — this document explains how the QC statistics feed an *alternative* gate but does not re-derive the dispatch. Second, the per-state process-noise covariance $\boldsymbol{Q}$ (clock random walk, ZWD, ambiguity, IMU bias) is purely a *dynamics*-side quantity and lives with the smoother (the process-noise section); everything below is measurement noise only.

1.1 The stochastic-model dispatch

Every undifferenced observation (r, i, s) is assigned a scalar variance by a strategy object selected from the `StochasticModelType` enum (`include/config/GnssConfigTypes.hpp`, the four-way {`Constant`, `Elevation`, `SNR`, `SatelliteBlock`} set). The dispatch lives in `src/core/model/StochasticModel.cpp`; each variant returns a variance in m^2 that becomes one diagonal entry of $\boldsymbol{R}$. The three geometry-aware strategies share a common low-elevation guard: when $\sin\theta \le 10^{-6}$ the elevation amplification is suppressed and the un-amplified variance is returned, so a satellite at the horizon does not inflate to a numerically infinite weight (`computeElevation`). The `Constant` strategy has no elevation term at all — it returns its a-priori variance directly from the dispatch (`computeConstant`) and never reaches the guard.

(a) Constant. The trivial strategy returns the a-priori variance of unit weight unchanged,

$$\sigma^2_{r,i,s} = \sigma^2_{0,\mathrm{type}(s)}, \tag{1}$$

with σ_0 drawn from `filter.measurement_noise` (`computeConstant`). This is the reference against which the other strategies are multiplicative.

(b) Elevation. The classical elevation-dependent model scales the zenith-referenced variance by an inverse power of $\sin\theta$,

$$\boxed{\sigma^2_{r,i,s} = \sigma^2_{\mathrm{zen}} \left(\sin\theta^i_r\right)^{-2p_{\mathrm{type}}}, \qquad p_P = p_\Phi = 1.0.} \tag{2}$$

The shipped defaults of `filter.code_elevation_power` and `filter.phase_elevation_power` are both unity, so the delivered model is *symmetric*: code and phase alike follow the legacy $\csc^2\theta$ variance law (`computeElevation`). The exponent in code is $-2p_{\mathrm{type}}$ because the strategy works in variance, not σ: $\left(\sin\theta\right)^{-2p}$ is exactly $\left(\sin\theta^{-p}\right)^2$. Rows whose observable code does not begin with `L`

— everything but carrier phase, including Doppler and the empty-`obs_type` fallback paths — take the code power.

Both exponents are configurable per observable class, and the asymmetric setting $p_\Phi = 0.5$ is the one alternative with a physical argument behind it, so it is worth recording why it is an *option* rather than the default. It gives a $\csc\theta$ variance ($\csc^{1/2}\theta$ in σ), a markedly gentler amplification at low elevation. The argument for it is that phase noise near the horizon is dominated by carrier-loop SNR and by antenna phase-centre miscalibration, neither of which follows the $\csc^2\theta$ mapping that governs code multipath and pseudorange path-length error; and that the milder amplification keeps low-elevation phase inside the useful range of the standardised-residual cycle-slip test instead of deweighting it into irrelevance. The argument against it is comparability — the code branch, and every archived benchmark, run at unity — and that is the position the loader now ships (`GnssConfig::getPhaseElevationPower`, whose documentation carries the amplification ratios at a 10° mask).

(c) SNR (SIGMA-ε). When carrier-to-noise density is available the strategy uses the Hadás–Bosy variance-additive form [1]. Writing the measured (or synthesised) density as $\mathrm{C/N_0}$ in dB Hz,

$$\boxed{\sigma^2_{\mathrm{SNR}} = a^2 + b^2\, 10^{-\mathrm{C/N_0}/10},} \tag{3}$$

where a is the high-SNR floor ($\sigma \to a$ as $\mathrm{C/N_0} \to \infty$) and b scales the SNR-dependent contribution; both have separate code/phase values (`snr_params_.code_a/b`, `phase_a/b`). The *variance*-additive composition in Equation (3) is mathematically the correct combination of two independent noise sources. The legacy σ-additive alternative $\sigma = a + b\,10^{-\mathrm{C/N_0}/10}$ is explicitly rejected in `computeSnr` because squaring it produces an unphysical $2ab\,10^{-\mathrm{C/N_0}/10}$ cross-term that has no noise-process interpretation — it would couple the two contributions as if they were correlated when they are not.

When no `S...` observable is present the model does not switch functional form; instead it *synthesises* a density from elevation so the noise curve stays continuous across the boundary (`synthesizeCn0FromElevation`):

$$\mathrm{C/N_0}(\theta) = 50 - 15\,\big(1 - \max(0, \sin\theta)\big) \ \ [\mathrm{dB\,Hz}], \tag{4}$$

i.e. a 50 dB Hz zenith maximum dropping 15 dB to the horizon. The density is then clamped to the configured working range [`snr_floor_dbhz`, `snr_ceil_dbhz`]. Two refinements follow: on code rows a per-band scale s^2_{band} from the SatelliteBlock code table multiplies σ^2_{SNR} (wider-bandwidth signals such as L5/E5a get $\sim\sqrt{10}$ less correlator-noise σ, hence a factor $\sim 1/10$ in variance), while phase stays band-flat because PLL noise is governed by carrier-loop bandwidth, not chip rate. Finally the Wieser hybrid step pipes the SNR variance through Equation (2) so that elevation-correlated multipath — which pure SNR weighting under-weights, since the carrier loop tracks cleanly even while the range is corrupted — is still captured.

(d) SatelliteBlock. The most detailed strategy keys the zenith σ on the transmitting satellite's ANTEX block type, then applies observable-specific frequency-band and tracking-attribute scales (`computeSatBlock`):

$$\boxed{\sigma^2_{r,i,s} = \big(\sigma_{\mathrm{block}(i),\,\mathrm{type}}\, s_{\mathrm{band}}\, s_{\mathrm{attr}}\big)^2 \big(\sin\theta\big)^{-2p_{\mathrm{type}}}.} \tag{5}$$

Here $\sigma_{\mathrm{block}(i),\mathrm{type}}$ is looked up by substring-matching the satellite's ATX type string against `block_code_std_m` / `block_phase_std_m` (falling back to `default_code_std_m` / `default_phase_`

`std_m` when no block matches); s_{band} is the per-band factor keyed on the RINEX band character (`obs_type[1]`); and s_{attr} is the tracking-attribute factor keyed on `obs_type[2]` via a *per-system* table, because RINEX-3 attributes carry constellation-specific meaning (a `C` attribute is the noisy GPS C/A but the high-precision Galileo E1C pilot). The attribute lookup order is `[system][attr]` → `[default][attr]` → 1.0. Code and phase carry separate band/attribute tables (`SatBlockNoiseParams::ObservableScale`) since chip-rate calibration governs code while carrier-tracking calibration — a much smaller band gradient — governs phase. As with the other geometry-aware strategies the block variance is finally amplified by Equation (2); phase only reaches this path when `apply_to_phase` is set, otherwise it takes the flat `default_phase_std_m` route in the top-level dispatch.

All four strategies populate diagonal entries only; the undifferenced $\boldsymbol{R}$ is therefore block-diagonal, exactly the skeleton of the observation noise model. Structured off-diagonal correlation appears only after differencing (Section 1.5).

1.2 A-posteriori variance factor

The entire Kalman-side QC layer described in this and the following three sections is a *default-off* diagnostic. `computeResidualQc_` is called from `KalmanFilter::update` only when `estimation.enable_variance_factor` or `estimation.enable_data_snooping` is set, and the correlation-aware kernel of Section 1.5 additionally requires `estimation.enable_full_r_kernel`; all three default to false. With them off the filter's only outlier gate is the fixed $|s| > 6$ path of the gating section, the variance factor keeps its initialised value of unity, and none of the redundancy numbers, w-tests or minimal detectable biases below are computed at all.

Once an epoch has been updated, the post-fit residuals $\hat{\boldsymbol{v}}$ provide an internal-consistency check on whether the assumed $\boldsymbol{R}$ was correctly scaled. The a-posteriori *variance factor* (estimated variance of unit weight) is

$$\boxed{\hat{\sigma}_0^2 = \frac{\Omega}{\text{dof}}, \qquad \Omega = \hat{\boldsymbol{v}}^{\mathrm{T}} \boldsymbol{R}^{-1} \hat{\boldsymbol{v}} = \sum_i \frac{\hat{v}_i^2}{R_{ii}} \,\Bigg|_{\boldsymbol{R}\,\text{diag}}, \qquad \text{dof} = \operatorname{tr} \boldsymbol{\Theta},} \tag{6}$$

where Ω is the weighted sum of squared residuals and the degrees of freedom dof is the trace of the redundancy matrix $\boldsymbol{\Theta}$ (Equation (11) below), i.e. the *effective* redundancy of a recursively-filtered epoch rather than the naive $m - n$ of a single batch (`computeResidualQc_`, `src/core/filter/KalmanFilter.cpp`). Because $\boldsymbol{R}$ is carried in absolute units, the a-priori variance of unit weight is exactly $\sigma_0^2 = 1$; a well-scaled model therefore yields $\hat{\sigma}_0^2 \approx 1$, with $\hat{\sigma}_0^2 \gg 1$ signalling under-weighted (optimistic) noise and $\hat{\sigma}_0^2 \ll 1$ pessimistic noise.

The variance factor is a *reporting-only* rescale: it never feeds back into the propagated covariance $\boldsymbol{P}$. When `enableVarianceFactor()` is set, the filter *logs* a copy of the position σ scaled by $\sqrt{\hat{\sigma}_0^2}$, i.e. $\sigma_{\text{pos}} \to \sigma_{\text{pos}} \sqrt{\hat{\sigma}_0^2}$, leaving the filter state untouched (`validatePostFitResiduals`, `src/core/filter/KalmanFilter.cpp`).

The same quantity has a batch incarnation in the normal-equation accumulator of the PPP NEQ-export path (constructed only under `enableNeqExport()` in `Application::run`). Given normal matrix $\boldsymbol{N}$, right-hand side $\boldsymbol{u}$, and the pre-reduced $\boldsymbol{l}^{\mathrm{T}}\boldsymbol{P}\boldsymbol{l}$ term, the accumulator forms (`NeqAccumulator::solve`)

$$\Omega = \boldsymbol{l}^{\mathrm{T}}\boldsymbol{P}\boldsymbol{l} - \boldsymbol{u}^{\mathrm{T}}\hat{\boldsymbol{x}}, \qquad \text{dof} = n_{\text{obs}} - (n_{\text{params}} + n_{\text{local}}), \tag{7}$$

$$\hat{\sigma}_0^2 = \frac{\Omega}{\text{dof}}, \qquad \sigma_i = \sqrt{\hat{\sigma}_0^2 \left(\boldsymbol{N}^{-1}\right)_{ii}}. \tag{8}$$

Here $\hat{\boldsymbol{x}} = \boldsymbol{N}^{-1}\boldsymbol{u}$ is the least-squares solution, the Schur-reduced local parameters n_{local} are subtracted from the redundancy alongside the retained parameters n_{params}, and each retained parameter's formal σ is the $\hat{\sigma}_0^2$-scaled square root of the inverse normal matrix diagonal — the batch analogue of Equation (6)'s recursive dof.

Neither of these is the network producer's *operational* variance factor. The undifferenced network estimator does not form Ω/dof at all; it estimates a robust unit-weight σ as a scaled median-absolute-deviation over the weighted phase-leveling residual rows $\tilde{v}_i = |\hat{v}_i|\sqrt{w_i}$,

$$\hat{\sigma}_{0,\text{net}} = \max\big(\texttt{raim_sigma0_floor},\ 1.4826\ \text{median}_i\,\tilde{v}_i\big), \tag{9}$$

with the 1.4826 consistency factor recovering a Gaussian σ from the MAD and `raim_sigma0_floor` (default 1.0) flooring it. Unlike the Kalman variance factor of Equation (6), $\hat{\sigma}_{0,\text{net}}$ is emphatically *not* reporting-only: it drives a within-epoch RAIM adaptation loop whenever it exceeds `network.raim_sigma0_trigger`, a whole-epoch and per-constellation product withhold above `network.raim_withhold_sigma0`, and the scale on the emitted URA. Those three responsibilities live in three separate seams: `networkSolveGlue` (`filter/NetworkSolveGlue`) owns the within-epoch loop and both withhold gates, the per-constellation one through `NetworkEstimator::decidePerConstWithhold`; `networkPostFitRaim` (`filter/NetworkRaim`) owns the post-fit pass and the exponentially-weighted fold of $\hat{\sigma}_{0,\text{net}}$ into the emitted URA scale; and `networkSolPrep` (`filter/NetworkSolPrep`) owns the emission-side prune that actually removes a withheld constellation's satellites. Sections 1.6 and 1.7 develop the adaptation loop and the pooled statistics; the emitted-product side is not treated here.

A third, non-operational variance-factor instrument exists on the network path. `NetworkCarryVce` estimates Floor and Rate variance components of the carry-row stochastic model by residual-form least-squares variance-component estimation at fixed weight, purely to check the hand-set carry floors against a measured $\hat{\sigma}$. It is default-off (`network.carry_vce`), never feeds back into the solve, and the feedback path `network.carry_vce_apply` is a hard configuration-load error — the class has no apply method at all.

1.3 Redundancy numbers, the Baarda w-test, and the MDB

The detection theory rests on the post-fit residual covariance $\boldsymbol{Q}$ (the covariance of $\hat{\boldsymbol{v}}$),

$$\boldsymbol{Q} \;=\; \boldsymbol{R} - \boldsymbol{H}\,\boldsymbol{P}^{+}\,\boldsymbol{H}^{\mathrm{T}}, \tag{10}$$

where $\boldsymbol{P}^{+}$ is the *updated* (post-fit) state covariance, so that $\boldsymbol{Q}$ is the covariance of $\hat{\boldsymbol{v}}$ propagated through the gain. Its diagonal $\boldsymbol{Q}_{ii}$ is built per row in `KalmanFilter::update` as $R_{ii} - (\boldsymbol{H}\boldsymbol{P}^{+}\boldsymbol{H}^{\mathrm{T}})_{ii}$; the full matrix is formed only under the off-diagonal kernel of Section 1.5.

Redundancy number. The (local) redundancy of observation i is

$$\boxed{r_i \;=\; \frac{\boldsymbol{Q}_{ii}}{R_{ii}} \;=\; \big(\boldsymbol{\Theta}\big)_{ii}, \qquad \boldsymbol{\Theta} = \boldsymbol{Q}\,\boldsymbol{R}^{-1}, \qquad r_i \in [0,1],} \tag{11}$$

clamped to $[0,1]$ for numerical safety in both branches of `computeResidualQc_`. $r_i \to 1$ means the observation is fully checked by the geometry (a blunder lands almost entirely in the residual); $r_i \to 0$ means it is *uncheckable* (a blunder is absorbed into the estimate and leaves no residual signature). The sum $\text{dof} = \sum_i r_i = \operatorname{tr}\boldsymbol{\Theta}$ is the effective number of degrees of freedom used in Equation (6).

The network producer reimplements Equation (11) in the batch/arrowhead algebra of the Schur-conditioned shared covariance, $r_i = 1 - w_i\,\boldsymbol{a}_i^{\mathrm{T}}\,\mathrm{Cov}(\boldsymbol{x})\,\boldsymbol{a}_i$ with $\boldsymbol{a}_i = [\,\boldsymbol{a}_{s,i} \mid \boldsymbol{a}_{r,i}\,]$ and the same $[0,1]$

clamp — algebraically the batch twin of $r_i = \boldsymbol{Q}_{ii}/R_{ii}$ (`NetworkNormalEquations::rowRedundancy`, gated by `network.raim_redundancy_normalize`, default off). Rows falling below the `kRedFloor` redundancy floor are dropped from the $\hat{\sigma}_{0,\text{net}}$ pool of Equation (9) as uncheckable — the network analogue of the KF snooping gate's `kUncheckableRedundancy` cutoff below, at a much smaller constant because it guards only the $\sqrt{r_i}$ standardisation, not a reset decision.

A structurally *exact* instance of $r_i = 0$ governs the network producer's entire phase-side QC design, and it is worth stating as a theorem rather than as a numerical caveat. Whenever each (station, satellite, signal) arc carries its own free ambiguity — the configuration in force whenever `network.phase_leveling` is off — every phase row is fitted *exactly* by its own parameter. The post-fit phase residual then vanishes identically, r_i is exactly zero, and no phase fault is detectable from the residuals at any magnitude: an undifferenced network with per-arc float ambiguities carries *no* phase-side redundancy whatsoever. Only when the ambiguities are stiffened by a carried prior do the phase rows acquire redundancy, and therefore fault-detection power — which is why the producer's phase RAIM (Section 1.6) is gated on phase-leveling in the first place, rather than merely working less well without it. The measure-only instrument that reports the collapse is `networkPostfitSigma0` (`filter/NetworkFrameDiag`), whose pooled phase variance factor is identically zero in that configuration; the zero is the finding, not a defect. Its pools deliberately exclude two row classes on the same reasoning: rows touching a shared spherical-harmonic column, which are pseudo-observations rather than data (Section 1.8), and the carry-coupling rows, which are a statement about the carry rather than about the epoch's measurements.

w-test. Baarda's data-snooping statistic standardises each residual by its own post-fit standard deviation. The diagonal (uncorrelated) form is

$$w_i \;=\; \frac{\hat{v}_i}{\sqrt{\boldsymbol{Q}_{ii}}} \tag{12}$$

(the diagonal branch of `computeResidualQc_`), which — crucially — divides by $\sqrt{\boldsymbol{Q}_{ii}}$, the *true* residual standard deviation, not by $\sqrt{R_{ii}}$. This is the Studentised residual proper; under the null hypothesis $w_i \sim \mathcal{N}(0,1)$, whereas the s used by the fixed gate floors the denominator at R and is therefore conservative. When the residual covariance is non-diagonal the correlated Teunissen generalisation is used instead (the `use_full` branch of the same routine):

$$\boxed{w_i \;=\; \frac{\left(\boldsymbol{Q}^{-1}\hat{\boldsymbol{v}}\right)_i}{\sqrt{\left(\boldsymbol{Q}^{-1}\right)_{ii}}}.} \tag{13}$$

Equation (13) reduces to Equation (12) when $\boldsymbol{Q}$ is diagonal, but accounts for the off-diagonal correlation among residuals introduced by differencing [2, 3].

Minimal detectable bias. The smallest blunder on observation i that the w-test will detect, at the configured significance and a conventional 80% test power, is

$$\boxed{\text{MDB}_i \;=\; \frac{\sigma_i\,\delta_0}{\sqrt{r_i}}, \qquad \delta_0 = \texttt{dataSnoopingCritical()} + 0.8416,} \tag{14}$$

with $\sigma_i = \sqrt{R_{ii}}$ the a-priori observation σ (both branches of `computeResidualQc_`). The non-centrality δ_0 is the critical value plus $\Phi^{-1}(0.80) = 0.8416$ (the 20% false-negative quantile, i.e. $\beta = 0.20$ power); for the default critical $k = 3.29$ this gives $\delta_0 \approx 4.13$. The $1/\sqrt{r_i}$ factor is the

geometric leverage: a poorly-checked observation ($r_i \to 0$) has an arbitrarily large MDB — the code returns $+\infty$ when r_i falls below `kRedFloor` — so a blunder there is by construction undetectable from the residual alone. The MDB is a Kalman-filter QC quantity only: the network producer implements no minimal detectable bias, relying instead on the robust $\hat{\sigma}_{0,\text{net}}$ RAIM loop of Equation (9).

1.4 Data-snooping gate override

The QC statistics are not merely reported; when `enableDataSnooping()` is set they drive an *alternative, parallel* outlier gate that can reselect the row the fixed $|s| > 6$ path missed. Both gates share the same downstream dispatch inside `KalmanFilter::validatePostFitResiduals`; only the *statistic, index, and critical value* differ. By default the gate is seeded from the fixed-σ triple $(\max_i |s_i|, \arg\max, 6)$ so the dispatch is byte-identical when snooping is off.

When snooping is enabled the gate is instead reseeded from the w-test over *checkable* rows only: rows whose redundancy falls below the `kUncheckableRedundancy` cutoff are skipped, because their residual is unreliable and resetting the corresponding ambiguity would discard good information. (The same cutoff screens the summary log, so what is reported and what is gated agree.) Over the surviving rows the gate fires on

$$\max_{\{i\,:\,r_i \geq \texttt{kUncheckableRedundancy}\}} |w_i| \;>\; \texttt{dataSnoopingCritical()}, \tag{15}$$

and the worst checkable $|w_i|$ row becomes the handled index. Because w_i uses the true residual σ (Equation (12)) rather than the R-floored s, this gate can catch a blunder on a high-leverage row whose floored s stayed below 6, and it overrides the s pick: the downstream Melbourne–Wübbena tiebreaker still operates in the σ domain, so when the w-test selected a row the σ cut missed, no σ-candidate is collected and the w-selected index is handled directly. This is the complement of the fixed standardised-residual path in the gating section, which this document does not re-derive.

1.5 Correlation-aware full-$\boldsymbol{R}$ kernel

The diagonal forms above assume $\boldsymbol{R}$ and $\boldsymbol{Q}$ are diagonal, which holds for undifferenced PPP. For BSSD and baseline-DD epochs the differencing operator $\boldsymbol{D}$ of the differencing transformations maps the diagonal undifferenced covariance into a *dense* one, $\boldsymbol{R}_{\text{SD/DD}} = \boldsymbol{D}\,\boldsymbol{R}_{\text{ZD}}\,\boldsymbol{D}^{\mathrm{T}}$, carrying the structured SD/DD cross-covariance between observations that share a pivot or reference. Under the off-diagonal kernel (`estimation.enable_full_r_kernel`, default off, which additionally requires that a dense $\boldsymbol{R}$ was actually supplied for the epoch — the `dense_r_supplied` test in `KalmanFilter::update`) the full residual covariance Equation (10) is built densely and symmetrised, and the QC quantities follow without approximation:

$$\boldsymbol{\Theta} = \boldsymbol{Q}\,\boldsymbol{R}^{-1}, \qquad r_i = \big(\boldsymbol{\Theta}\big)_{ii}, \qquad \text{dof} = \operatorname{tr}\boldsymbol{\Theta}, \tag{16}$$

$$\Omega = \hat{\boldsymbol{v}}^{\mathrm{T}}\boldsymbol{R}^{-1}\hat{\boldsymbol{v}}, \qquad w_i = \frac{\big(\boldsymbol{Q}^{-1}\hat{\boldsymbol{v}}\big)_i}{\sqrt{(\boldsymbol{Q}^{-1})_{ii}}}, \qquad \text{MDB}_i = \frac{\sqrt{R_{ii}}\,\delta_0}{\sqrt{r_i}}. \tag{17}$$

$\boldsymbol{R}^{-1}$ and $\boldsymbol{Q}^{-1}$ are obtained by LDL^{T} solves against the identity, exploiting the symmetric positive-definiteness of both. The kernel is consistent by construction: the diagonal of the full $\boldsymbol{\Theta}$ and of the full Teunissen w equal the per-row quantities of Section 1.3 whenever the off-diagonals vanish, so the *outlier-handling path is unchanged behind the flag* — it only gains correctness on differenced epochs where the residual correlations are real. Note the narrow scope of the flag: the gain and Joseph-form update already honour a dense $\boldsymbol{R}$ whenever one is supplied, independently of it; the flag decides only whether the *post-fit residual* covariance is formed densely. This is the measurement-side counterpart to the structured correlation the differencing section introduces on the model side.

1.6 Producer-side RAIM: within-epoch adaptation

The network producer's response to a tripped $\hat{\sigma}_{0,\mathrm{net}}$ is not a single exclusion but a choice of *granularity*, made from a robust two-way decomposition of the epoch's standardised residuals. Taking the largest standardised residual m_{ri} of each (station r, satellite i) carried-ambiguity cell as a sparse two-way table and applying a Tukey median polish,

$$m_{ri} \;\approx\; \mu \;+\; \alpha_r \;+\; \beta_i \;+\; \gamma_{ri}, \tag{18}$$

splits the contamination into a grand effect μ, a per-station row effect α_r, a per-satellite column effect β_i, and an isolated-arc interaction γ_{ri} (`medianPolish2Way`, `filter/NetworkRobustStats`). Sweeping medians rather than means is what makes the split usable: a minority of very large cells — the actual outliers — does not distort the additive effects, so a bad satellite lighting up a whole column is separable from a bad station lighting up a whole row, and genuinely isolated arc slips are left in the interaction term.

Each effect is a candidate culprit, and each must clear effect$/\hat{\sigma}_{0,\mathrm{net}} > k_{\mathrm{RAIM}}$ against the configured gate (`network.raim_k_sigma`) to be eligible at all. The three are then ranked surgical-first: on equal score an arc beats a satellite beats a station, and the station branch is additionally de-rated by a bias factor (`network.raim_station_drop_bias`, active under `network.raim_surgical_station`) so the coarsest response — which discards good arcs along with bad — is taken only when it dominates by that margin. A consensus rule short-circuits arc-by-arc peeling: when at least `network.raim_min_stations` of one satellite's arcs are over the gate in a single iteration, the arc candidate is redirected to the whole satellite, because a corrupted *shared* clock or phase bias is a network-wide fault rather than a stack of independent local ones.

The response itself distinguishes a bad *measurement* from a stale *parameter*, and that distinction is the load-bearing design decision. A whole-station culprit is excluded — its data are bad, and deweighting or exclusion is the right tool for bad data. A satellite or arc culprit, by contrast, is overwhelmingly a cycle slip, and a slipped ambiguity does not need less weight, it needs a new *value*. The epoch is therefore replayed over a freshly rebuilt normal-equation system with the culprit's ambiguities *refloated*: their measurements retained at full weight, so the local block stays positive-definite and no drop-induced singularity is introduced, but their carried prior withheld, so the re-solve is free to find the new value and absorb the slip locally. The loop iterates until $\hat{\sigma}_{0,\mathrm{net}}$ falls below the trigger or a small fixed pass cap is reached; only the rare, already-contaminated epochs pay for a re-solve, and any failure reverts to the last good solution (`networkSolveGlue`).

Arming honesty: `network.raim` and `network.raim_within_epoch` both default to false, and *both* tiers are additionally gated on the phase-leveling carry being warm — the within-epoch loop inside `networkSolveGlue`, the post-fit pass at the `networkPostFitRaim` call site. With `network.phase_leveling` off there are no carried ambiguities to refloat and, by the redundancy-collapse argument of Section 1.3, no phase-side redundancy to test at all, so the producer's RAIM is inert by construction rather than by policy.

1.7 Pooled variance factors and the per-constellation carve-out

Equation (9) pools only the carried-ambiguity *phase* rows. Two extensions widen or partition that support without altering the estimator; both are default-off.

Code and iono-constraint pools. In the historical arrangement the code rows and the per-line-of-sight $\leftrightarrow$ spherical-harmonic iono-constraint rows are screened by no tier whatsoever. Under

`network.raim_we_code_iono` the same scaled-MAD estimator is applied to each of them over its own disjoint row support,

$$\hat{\sigma}_{0,\text{code}} = 1.4826\ \text{median}_{i\in\mathcal{C}}\,\tilde{v}_i, \qquad \hat{\sigma}_{0,\text{iono}} = 1.4826\ \text{median}_{i\in\mathcal{I}}\,\tilde{v}_i, \tag{19}$$

each with its own trigger, and each row assigned to exactly one of the three supports. Neither pool carries the floor of Equation (9), and neither displaces the phase pool as the driver of the whole-epoch ceiling. The adaptation gains two response classes matched to what these rows can actually diagnose: an epoch-scoped station exclusion, and a network-wide *deweighting* of one satellite's code rows — a fallback, not a drop, in keeping with the project's standing preference for degrading a measurement over discarding it.

Per-constellation carve-out. When the global ceiling trips, the default response withholds the whole epoch's product. Under `network.per_const_withhold` the same pool is instead partitioned by the ambiguity's satellite constellation and each part judged on its own,

$$\hat{\sigma}_{0,c} = \max\big(\texttt{raim_sigma0_floor},\ 1.4826\ \text{median}\{\tilde{v}_i : i \in \mathcal{P}_c\}\big), \tag{20}$$

over exactly the rows $\mathcal{P}_c$ that the global statistic pools — same statistic, same ceiling, restricted support. A constellation is published if and only if $\hat{\sigma}_{0,c}$ clears that ceiling *and* $|\mathcal{P}_c|$ meets a minimum row count, since too few rows cannot certify cleanliness inside a contaminated epoch (`NetworkEstimator::decidePerConstWithhold`).

The approximation is stated rather than hidden. The residuals entering Equation (20) are evaluated at the *joint* solution, so a contamination in one constellation perturbs the shared per-station nuisances (receiver clock, tropospheric delay, slant ionosphere) and leaks into another constellation's residuals; and each row's redundancy is its share of the joint system's redundancy, not of a hypothetical per-constellation solve. $\hat{\sigma}_{0,c}$ therefore under-separates cross-constellation coupling, and the outcome "every constellation individually clean while the global statistic tripped" is the honest edge of that approximation — it is published with a warning and counted, not silently absorbed. The rank and observability gate is deliberately left with whole-epoch semantics: a rank deficiency invalidates the declared datum itself, so the product would be a meaningless least-norm split and no healthy subset exists to carve out.

1.8 Station-relative deweighting of the ionosphere constraint

A single-layer spherical-harmonic VTEC surface structurally cannot represent per-line-of-sight slant structure: the large majority of LOS residuals against the fitted field exceed 3σ at any degree and at every network scale. That exceedance is a chronic, roughly common-mode background, so an *absolute* 3σ gate would engage every station and diagnose nothing. The genuinely hostile sites — equatorial-anomaly and scintillation-belt stations whose true slant ionosphere lies beyond the field's representability — stand out only as a *relative* excess. That is the reason the gate below is network-relative and robust rather than absolute (`NetworkIonoDeweightGate`).

Each station's per-epoch misfit statistic m_r is first smoothed by an exponentially-weighted moving average with forgetting factor $\phi = 2^{-1/W}$, i.e. a half-life of W epochs, then standardised against the network's own robust centre and scale. Writing $\mathcal{M}$ for the set of mature, non-statically-listed stations,

$$z_r \;=\; \frac{\bar{m}_r - \text{med}}{1.4826\,\text{MAD}}, \qquad \text{med} = \text{median}_{r'\in\mathcal{M}}\,\bar{m}_{r'}, \qquad \text{MAD} = \text{median}_{r'\in\mathcal{M}}\big|\bar{m}_{r'} - \text{med}\big|. \tag{21}$$

The median centring removes the common-mode background; the MAD scale tolerates a poison minority, its 50% breakdown point being precisely the property an absolute threshold lacks. A minimum mature population is required before the robust centre is formed at all, and statically-listed stations are excluded from it so that a known-bad site cannot mask the detection of others.

Detection is hysteretic — a station engages at $z_r > \sigma_{\mathrm{hi}}$ and releases at $z_r < \sigma_{\mathrm{lo}}$, with σ_{lo} forced below σ_{hi} — and while engaged the target weight follows a smooth IGG-style ramp, unity at the release threshold and equal to a configured floor $w_{\min}$ at and above the engage threshold, quadratic between:

$$\boxed{w_r^* = w_{\min} + (1 - w_{\min})\,(1 - t)^2, \qquad t = \frac{\mathrm{clamp}\big(z_r,\, \sigma_{\mathrm{lo}},\, \sigma_{\mathrm{hi}}\big) - \sigma_{\mathrm{lo}}}{\sigma_{\mathrm{hi}} - \sigma_{\mathrm{lo}}}.} \tag{22}$$

The realised weight slews toward w_r^* by at most a fixed increment per clean epoch and is clamped to $[w_{\min}, 1]$, which keeps the gate from oscillating against its own effect on the field fit.

The action is purely on $\boldsymbol{R}$. The factor w_r multiplies the *information* of that station's spherical-harmonic $\leftrightarrow$ slant-iono pseudo-observation rows, equivalently $\sigma^2 \mapsto \sigma^2 / w_r$. Because those rows are one joint pseudo-observation coupling the station's per-LOS slant ionosphere to the shared field in both directions, a single w_r relaxes both directions uniformly; $w_r \to 0$ decouples that station's ionosphere entirely — it then floats on its own geometry-free combination, still observable, so no rank hole opens — while the station continues to serve the shared clocks, biases, datum graph and position. It is the surgical alternative to dropping a hostile station, and it is a weighting change only: no state is removed and no datum is altered.

Two honesty notes. Datum-change epochs, pivot handover included, are excluded outright — the gate is neither fed nor advanced on them, because their residual spikes are datum artefacts rather than station misfit — so every weight is simply held across them. And the adaptive path is default-off (`network.iono_constraint_deweight_adaptive`); with it off and the static station list empty, every w_r is unity and the producer is byte-identical to the ungated build. The gate is advanced once per epoch from the serial post-solve harvest in `networkShHealth` (`filter/NetworkShHealth`) and read (const, race-free) during the parallel per-station assembly.

1.9 State-domain emission gates

Everything above tests *residuals* against $\boldsymbol{R}$. The producer also carries two gates that test the *solved state* and act on the emitted product rather than on the estimation. A reader who meets them as "QC" should know both that they are a different kind of test and where they live — in neither case the seam whose name most suggests it.

Spherical-harmonic field health. Under `network.sh_health_gate` (default off) the solved VTEC coefficients are read back out of the shared solution, the dual-layer degree-zero terms are summed to a total mean VTEC, and that value is tested against a physically admissible band together with a ceiling on the RMS of all coefficients. An out-of-band epoch must repeat for a configured number of consecutive epochs before the gate engages, which debounces the single-epoch excursion. The verdict is *emission-only* by construction: the solve, the carry stash and every QC tier continue to run while emission is withheld, so an era lasting hours cannot expire the carry the way a skip-epoch response would. A failed coefficient lookup leaves the gate state unchanged rather than fabricating a release. The gate has no release-or-degrade policy — a sick era withholds for its full duration — and that is an open policy item, not a design conclusion. The verdict is formed in `networkSolveGlue`, the only pre-emission seam holding the solved shared vector; `networkShHealth`

(`filter/NetworkShHealth`) owns the health *probe*, the deweight fold of Section 1.8 and the carry variance-component update, not this verdict.

Clock-runaway quarantine. Under `network.clock_runaway_check` (default off) the epoch's satellite clock solutions are grouped by constellation, and a constellation is dropped from the emission when the median of $|\text{clock} - \text{median}_{\text{all}}|$ exceeds a magnitude ceiling, or the median |drift| exceeds a rate ceiling. Like the carve-out of Section 1.7, this degrades scope rather than the whole product. The code documents an inversion hazard instead of defending against it: the magnitude test is referenced to the global median over *all* clock satellites, so if the runaway constellation supplies half or more of them the comparison inverts — the healthy minority trips the ceiling and is quarantined while the runaway passes. The single-constellation degenerate case is warned at configuration load. The quarantine lives in `networkSolPrep` (`filter/NetworkSolPrep`); `networkClockQc` (`filter/NetworkClockQc`) is a different tier entirely — the two-state oscillator model with its normalised-innovation fault test, the decoupled emit-clock-rate estimator, and the per-satellite carried-clock sanity monitor.

References

[1] J. Sanz Subirana, J. M. Juan Zornoza, and M. Hernández-Pajares. *GNSS Data Processing, Volume I: Fundamentals and Algorithms*. ESA TM-23/I. Noordwijk, The Netherlands: ESA Communications, 2013.

[2] P. J. G. Teunissen. "Generalized inverses, adjustment, the datum problem and S-transformations". In: *Optimization and Design of Geodetic Networks* (1985), pp. 11–55.

[3] P. J. G. Teunissen and O. Montenbruck, eds. *Springer Handbook of Global Navigation Satellite Systems*. Cham, Switzerland: Springer, 2017. ISBN: 978-3-319-42926-7. DOI: 10.1007/978-3-319-42928-1.